

Implementacja algorytmu równoległego sortowania kubelkowego

Robert Mesek
Olaf Fertig

13 maja 2026

1 Wstęp

1.1 Cel ćwiczenia

Celem niniejszego ćwiczenia było zapoznanie się z mechanizmami ochrony pamięci współdzielonej między wątkami w standardzie OpenMP oraz praktyczna ocena kosztu ich wykorzystania. Problem badawczy zrealizowano poprzez implementację równoległego algorytmu sortowania kubelkowego (ang. *Bucket Sort*) w dwóch odmiennych wariantach. Głównym zadaniem było zbadanie, jak różne podejścia do podziału danych, unikania wyścigów danych (ang. *data race*) oraz redundancji operacji wejścia/wyjścia wpływają na przyspieszenie obliczeń (ang. *speedup*) i ogólną wydajność algorytmu w środowisku wielowątkowym.

1.2 Badane algorytmy

Sortowanie kubelkowe polega na rozdzieleniu sortowanych elementów do mniejszych przedziałów (kubeków), a następnie niezależnym posortowaniu każdego z nich. W ramach ćwiczenia zbadano dwa podejścia do zrównoleżenia tego procesu:

- **Wariant 1:** Przestrzeń dostępnych kubków jest dzielona pomiędzy wątki. Każdy wątek iteruje po całej tablicy początkowej (odczyt redundantny), ale zapisuje liczby wyłącznie do swoich prywatnych kubków. Dzięki temu nie występują konflikty przy zapisie. Każdy wątek niezależnie sortuje swoje kubki i przepisuje je do odpowiedniego, wyznaczonego dla siebie fragmentu tablicy wynikowej.
- **Wariant 3:** Tablica początkowa jest dzielona na równe, rozłączne fragmenty przypisane do poszczególnych wątków, co eliminuje redundancję odczytów. Każdy wątek posiada jednak pełną pulę kubków. Po rozdzieleniu przypisanych liczb do swoich lokalnych kubków, wątki muszą wspólnie połączyć (złączyć) kubki o tych samych zakresach zlokalizowane w różnych wątkach, a dopiero potem posortować powstałe, globalne kubki.

1.3 Generacja liczb losowych i testy pętli for

Jako strukturę danych reprezentującą wektor liczb wykorzystano jednowymiarową tablicę typu `double` alokowaną na sterście z wykorzystaniem funkcji `malloc(array_size * sizeof(double))`. W ramach badań porównano dwa generatory liczb pseudolosowych: systemową funkcję `rand_r()` oraz autorski algorytm `xorshift32`. Implementacja szybkiego generatora `xorshift32` prezentuje się następująco:

```
uint32_t xorshift32(uint32_t* state) {  
    // algorithm "xor" from p. 4 of Marsaglia, "Xorshift RNGs"  
    uint32_t x = *state;  
    x ^= x << 13;  
    x ^= x >> 17;  
    x ^= x << 5;  
    return *state = x;  
}
```

Testy przeprowadzono dla rozwiązania sekwencyjnego oraz wersji równoległych OpenMP testujących różne parametry klauzuli `schedule` (`static`, `dynamic`, `guided`, `auto`). Eksperymenty uruchomiono na 12-rdzeniowym procesorze z ograniczeniem do 12 wątków (`OMP_NUM_THREADS=12`), a dla wiarygodności pomiarów wykorzystano uśrednienie z 10 powtórzeń.

Poniższe tabele prezentują czasy wykonania wyłącznie dla optymalnego podziału, w którym rozmiar porcji (*chunk*) wynosi `loop_count/number_of_threads`.

Mały problem ($N = 100$, `chunk_size = 8`)

Funkcja	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	0.000442	0.000425	0.000448	0.000008
omp_static_rand_r	0.751968	0.662619	0.991346	0.103037
omp_dynamic_rand_r	0.004335	0.003770	0.004823	0.000320
omp_guided_rand_r	0.004714	0.004004	0.005185	0.000376
omp_auto_rand_r	0.002626	0.002279	0.003046	0.000245
sequential_xorshift32	0.000165	0.000138	0.000383	0.000077
omp_static_xorshift32	0.002356	0.002012	0.002715	0.000212
omp_dynamic_xorshift32	0.003410	0.002885	0.004014	0.000332
omp_guided_xorshift32	0.004681	0.003705	0.005430	0.000554
omp_auto_xorshift32	0.002381	0.002066	0.002736	0.000222

Średni problem ($N = 300\,000$, `chunk_size = 25\,000`)

Funkcja	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	2.130417	2.057283	2.362228	0.094652
omp_static_rand_r	0.842754	0.784223	0.914409	0.037919
omp_dynamic_rand_r	0.136884	0.099191	0.179387	0.033273
omp_guided_rand_r	0.130905	0.098083	0.187618	0.033522
omp_auto_rand_r	0.136827	0.097530	0.189151	0.032436
sequential_xorshift32	0.390308	0.353398	0.449224	0.044399
omp_static_xorshift32	0.092400	0.090674	0.093985	0.000912
omp_dynamic_xorshift32	0.053726	0.032794	0.086978	0.014516
omp_guided_xorshift32	0.054310	0.038756	0.076512	0.013199
omp_auto_xorshift32	0.051153	0.032091	0.080356	0.016743

Duży problem ($N = 1\,000\,000\,000$, `chunk_size = 83\,333\,333`)

Funkcja	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	8093.181217	7887.736839	8326.469586	160.348006
omp_static_rand_r	711.812622	334.205901	1735.168439	576.374964
omp_dynamic_rand_r	409.348204	334.532024	934.999079	188.840301
omp_guided_rand_r	394.439557	334.232114	908.091583	180.493235
omp_auto_rand_r	360.636151	333.430730	565.272663	71.933819
sequential_xorshift32	1182.602893	1174.265823	1210.079435	10.408489
omp_static_xorshift32	297.007404	288.205638	315.587334	7.947635
omp_dynamic_xorshift32	300.199296	279.677135	326.731428	15.235039
omp_guided_xorshift32	291.042016	274.893820	306.875936	10.795116
omp_auto_xorshift32	293.130166	279.560436	306.985462	9.259098

Wnioski

Na podstawie zebranych pomiarów sformułowano następujące wnioski dotyczące generatorów i podziału pętli:

- **Narzut współbieżności:** Dla bardzo małych rozmiarów tablic optymalna pozostaje wersja sekwencyjna, ponieważ czas potrzebny na zarządzanie wątkami OpenMP przewyższa zyski ze zrównoleglenia.
- **Wydażność xorshift32:** Implementacja algorytmu xorshift32 jest nie tylko znacznie szybsza od systemowego `rand_r()`, ale zapewnia również dużo wyższą stabilność wyników (mniejsze odchylenie standardowe).
- **Wpływ schedule:** Zbyt rozdrobiony podział (np. `schedule(dynamic, 1)`) przynosi najgorsze rezultaty przy dużych pętlach ze względu na olbrzymi koszt synchronizacji i przełączania iteracji pomiędzy wątkami. Z drugiej strony, wykorzystanie strategii `schedule(auto)` okazało się bardzo efektywnym wyborem w testowanym problemie.

1.4 Środowisko testowe i metodyka pomiarowa

Eksperymenty przeprowadzono na klastrze obliczeniowym Ares, wykorzystując węzły wyposażone w wielordzeniowe procesory. Algorytmy zaimplementowano w języku C z użyciem biblioteki OpenMP.

W celu zapewnienia wysokiej wiarygodności statystycznej wyników, testy przeprowadzono dla różnych wielkości problemu (od $N = 1\,000$ do $N = 100\,000\,000$ elementów) oraz różnej liczby dostępnych wątków (od 1 do 32). Dla każdej unikalnej konfiguracji parametrów (rozmiar tablicy, liczba kubeków, liczba procesorów) program uruchamiano 10-krotnie, za każdym razem z innym początkowym ziarnem (*seed*). Czas wykonania poszczególnych faz algorytmu (dystrybucja, łączenie, sortowanie, kopiowanie) oraz czas całkowity uśredniono ze wszystkich prób. Na podstawie zebranych danych wyznaczono miarę przyspieszenia ($S = T_1/T_p$) dla poszczególnych etapów algorytmu.

2 Wyniki i analiza algorytmów

2.1 Wariant 1 (Bucketsort V1)

Analiza wydajności na podstawie wykresów

Z uzyskanych wykresów (Rysunek 2) płyną bardzo wyraźne wnioski dotyczące zachowania poszczególnych faz algorytmu:

- **Faza sortowania (doskonała skalowalność):** Czas sortowania lokalnych kubeków bardzo stromo maleje wraz ze wzrostem liczby wątków. Na wykresie przyspieszenia widać, że ta faza skaluje się niemal idealnie liniowo do około 15 wątków, ostatecznie osiągając imponujące przyspieszenie rzędu $S \approx 24$ dla 32 wątków. Wynika to z faktu, że wątki sortują swoje prywatne zasoby w sposób całkowicie bezkonfliktowy.
- **Faza dystrybucji (wąskie gardło):** Czas dystrybucji pozostaje niemal stały (płaska linia w okolicach 1500-2000 ms), a jej przyspieszenie wynosi ≈ 1 , niezależnie od liczby uruchomionych wątków. Ogranicza to drastycznie całkowite przyspieszenie algorytmu, które zgodnie z prawem Amdahla stabilizuje się na maksymalnym poziomie $S \approx 7$.

Odpowiedzi na pytania problemowe (V1)

a) Czy potrzebna jest jakaś ochrona danych wspólnych (tablica początkowa: przy odczycie i przy zapisie; kubki: przy zapisie, sortowaniu kubka, odczycie)?

Nie, w wariantcie 1 **nie jest wymagana** żadna jawna ochrona danych wspólnych (np. dyrektywy `atomic`, `critical` czy `locki`).

- *Tablica początkowa (odczyt):* Równoległy odczyt z pamięci współdzielonej jest sprzętowo bezpieczny.

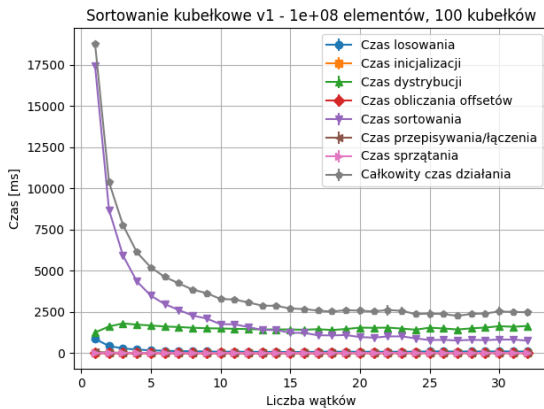
- *Tablica początkowa (zapis)*: Dzięki uprzedniemu obliczeniu unikalnych offsetów dla każdego wątku (faza `calc_offsets`), wątki zapisują posortowane wyniki do rozłącznych, niepokrywających się segmentów pamięci.
- *Kubelki*: Każdy wątek dynamicznie alokuje własne, prywatne kubelki na sterce wewnątrz bloku `#pragma omp parallel`. Zarówno zapis podczas dystrybucji, jak i późniejsze sortowanie i odczyt odbywają się lokalnie i bezkonfliktowo.

b) Czy zapewnienie, by wątki nie czytały tego samego elementu w tym samym czasie przyspieszy program?

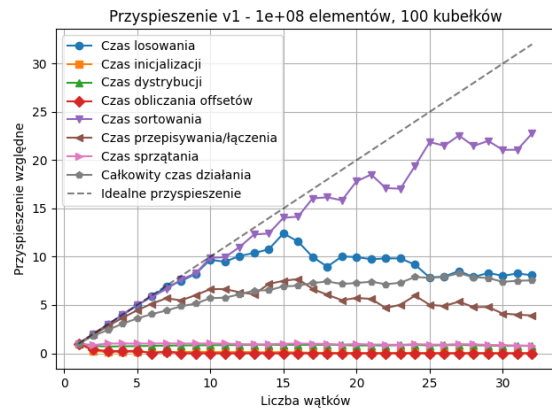
Zdecydowanie tak, co bezpośrednio udowadniają powyższe wykresy (płaska linia przyspieszenia dla fazy dystrybucji). W obecnej implementacji każdy z p wątków odczytuje pełną tablicę N elementów. Powoduje to powielenie operacji I/O, co przy 32 wątkach i stumilionowej tablicy oznacza konieczność pobrania z pamięci RAM ponad 12 GB danych ($32 \times 10^8 \times 4$ bajty). Prowadzi to do natychmiastowego nasycenia przepustowości szyny pamięci (zjawisko *memory wall*). Wyeliminowanie tej redundancji znacząco zmniejszyłoby czas dystrybucji.

c) Jaki jest rząd złożoności obliczeniowej algorytmu, a jaka jest praca algorytmu równoległego? Czy algorytm jest sekwencyjnie-efektywny?

- **Złożoność algorytmu sekwencyjnego**: Optymalny algorytm sekwencyjny sortowania kubelkowego posiada złożoność rzędu $O(N + N \log(N/K))$, gdzie N to liczba elementów, a K to liczba kubelków.
- **Praca algorytmu równoległego**: Praca $W(N)$, definiowana jako suma operacji wykonanych przez wszystkie wątki, wynosi $W(N) = O(p \cdot N + N \log(N/K))$. Czynniki $p \cdot N$ wynika z faktu, że każdy z p procesorów przegląda całą tablicę rozmiaru N podczas fazy dystrybucji.
- **Seqwencyjna efektywność**: Algorytm w wariancie 1 **nie jest sekwencyjnie-efektywny**. Aby algorytm był sekwencyjnie efektywny, jego całkowita praca $W(N)$ musi być asymptotycznie równa złożoności optymalnego algorytmu sekwencyjnego. W tym przypadku całkowita praca rośnie wraz z liczbą wątków ($p \cdot N$), co łamie tę zasadę i skutkuje marnowaniem zasobów obliczeniowych.

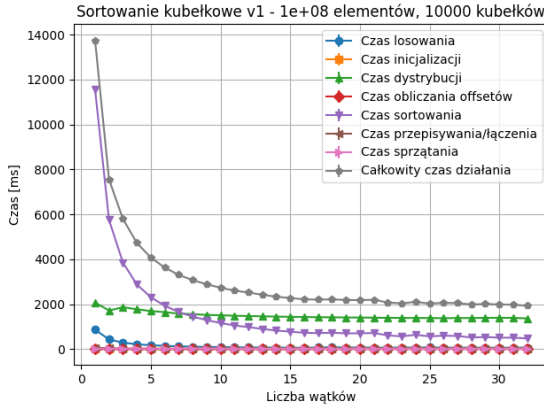


(a) Czas wykonania

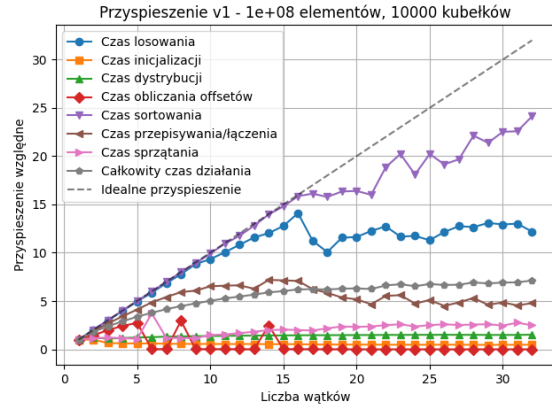


(b) Przyspieszenie

Rysunek 1: Analiza wydajności algorytmu w wersji 1. dla małego rozmiaru kubelka.

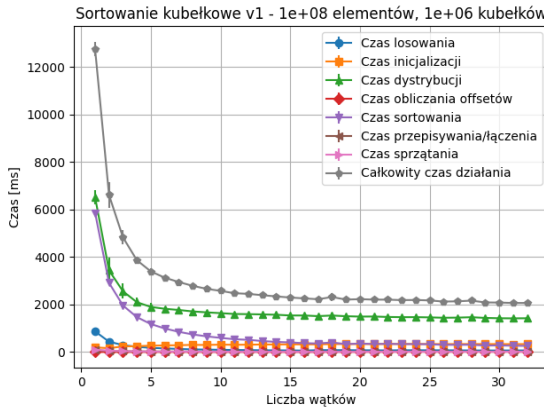


(a) Czas wykonania

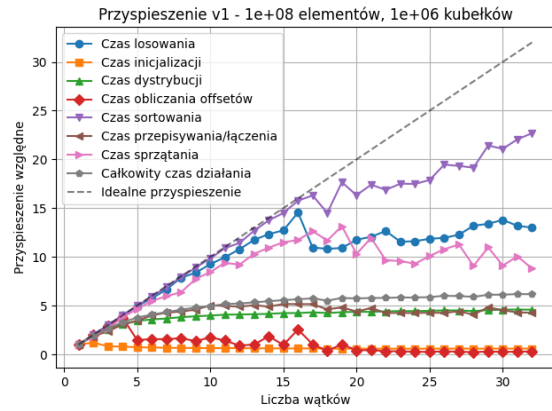


(b) Przyspieszenie

Rysunek 2: Analiza wydajności algorytmu w wersji 1. dla standardowego rozmiaru kubełka ($\sqrt{N}$).



(a) Czas wykonania



(b) Przyspieszenie

Rysunek 3: Analiza wydajności algorytmu w wersji 1. dla dużego rozmiaru kubełka.

2.2 Wariant 3 (Bucketsort V3)

Analiza wydajności na podstawie wykresów

Na podstawie wyników dla standardowego rozmiaru kubełka $\sqrt{N}$ (Rysunek 5) wysuwają się następujące spostrzeżenia:

- **Faza sortowania:** Faza ta wykazuje niemal idealne przyspieszenie, pokrywając się z linią teoretyczną aż do 32 wątków. Wynika to z faktu, że po fazie łączenia, każdy wątek sortuje przydzielony mu, rozłączny fragment tablicy wynikowej za pomocą funkcji `qsort`, co nie generuje żadnych konfliktów w dostępie do pamięci.
- **Fazy pomocnicze:** W przypadku faz, gdzie korzystamy z systemowej alokacji lub zwalniania pamięci nie skalują się z ilością wątków, a dodatkowo przy ponad 8 wątkach możemy zaobserwować wzrost czasu działania tych faz. Wskazuje to na bardzo nie korzystne zachowanie, gdzie występuje walka o systemowe zasoby. Oczywiście są pewne rozwiązania tego problemu, jak wykorzystanie pre-alokacji puli pamięci, lecz jest to poza zakresem naszej implementacji. Jedyną sekwencyjną częścią algorytmu jest znalezienie pozycji początkowej w tablicy wyjściowej, gdzie każdy wątek będzie wpisywał wartości ze swojego lokalnego kubełka. Aby oszczędzić pamięć w postaci tablicy wyjściowej posłuży tablica wejściowa podzielona na obszary globalnych kubełków. Są to rozłączne obszary, więc ostateczne sortowanie implementowane jest równoległe.

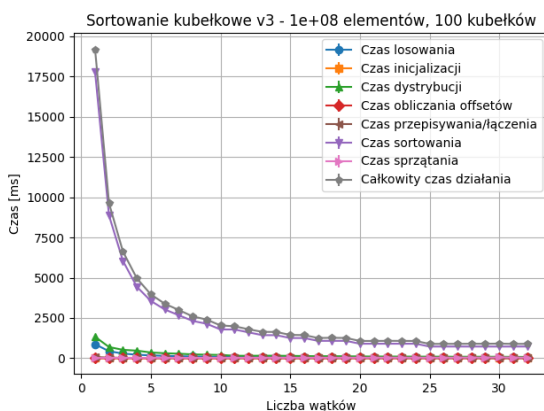
Odpowiedzi na pytania problemowe (V3)

a) Czy potrzebna jest jakaś ochrona danych wspólnych (tablica początkowa: przy odczycie i przy zapisie; kubełki: przy zapisie, łączeniu, sortowaniu kubełka, odczycie)?

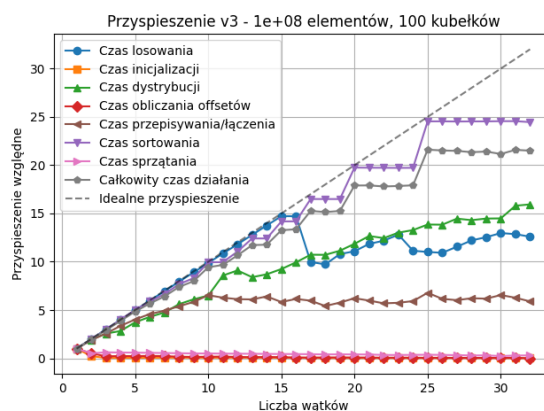
Implementacja wariantu 3 nie wymaga jakichkolwiek mechanizmów ochrony pamięci wspólnej. Wątki czytają rozłączne fragmenty tablicy. Każdy wątek operuje na własnej strukturze `local_buckets`. Dzięki obliczeniu `bucket_offsets`, każdy wątek zna dokładnie zakres tablicy głównej, do którego kopiuje dane ze swoich lokalnych kubełków bez nakładania się obszarów zapisu. Sortowanie gotowej tablicy odbywa się niezależnie dla każdego globalnego kubełka.

b) Jaki jest rząd złożoności obliczeniowej algorytmu, a jaka jest praca algorytmu równoległego, czy algorytm jest sekwencyjnie-efektywny?

- **Złożoność algorytmu sekwencyjnego:** Jak wcześniej, tzn. $O(N + N \log(N/K))$, gdzie N to liczba elementów, a K to liczba kubełków.
- **Praca algorytmu równoległego:** $W(N) = O(N + N \log(N/K) + p \cdot K)$. Czynniki $p \cdot K$ wynika z iterowania przez wszystkie K kubełków w każdym z p wątków podczas sumowania rozmiarów i scalania danych.
- **Seqwencyjna efektywność:** Algorytm w wariacie 3 jest bliski sekwencyjnie-efektywnemu. W przeciwieństwie do wariantu 1, całkowita liczba wykonanych operacji nie zależy od liczby wątków w odniesieniu do liczby elementów N , a jedynie od liczby kubełków $p \cdot K$. W tym przypadku K może pozostać stałą, lub skalować się znacznie wolniej $\approx \sqrt{N}$ w zależności od implementacji.

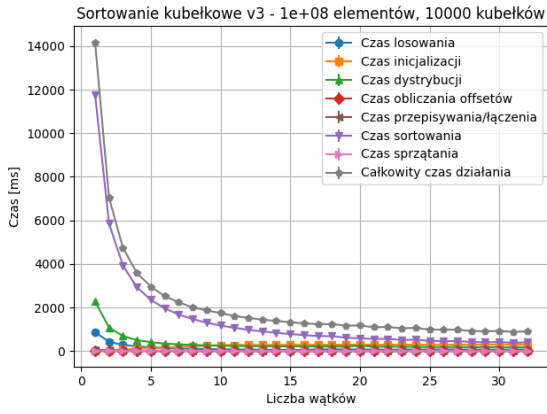


(a) Czas wykonania

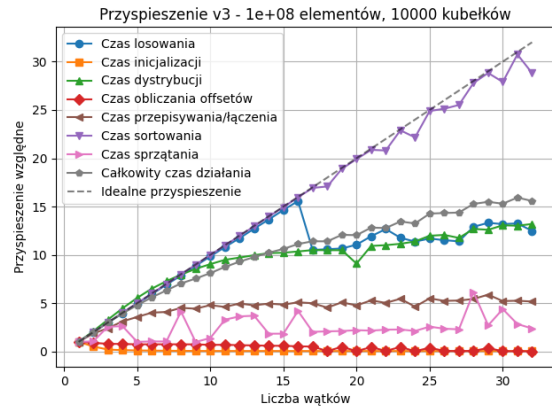


(b) Przyspieszenie

Rysunek 4: Analiza wydajności algorytmu w wersji 3. dla małego rozmiaru kubełka.

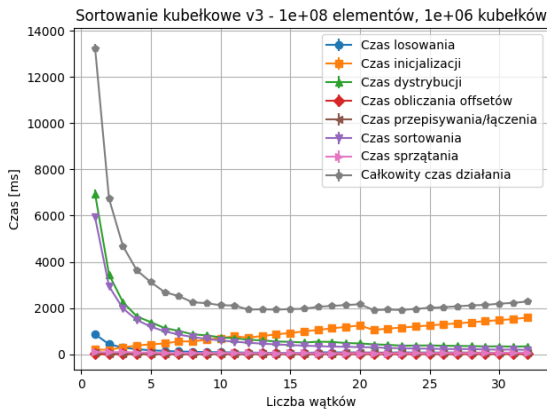


(a) Czas wykonania

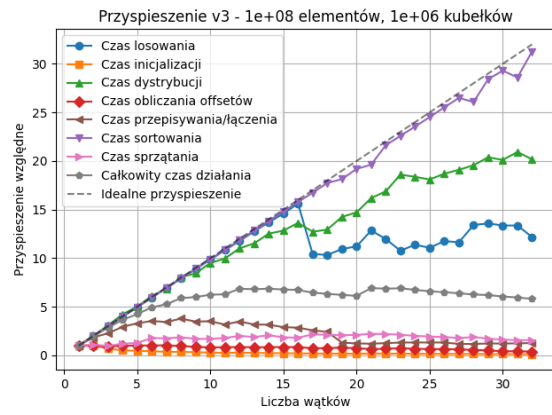


(b) Przyspieszenie

Rysunek 5: Analiza wydajności algorytmu w wersji 3. dla standardowego rozmiaru kubełka ($\sqrt{N}$).



(a) Czas wykonania



(b) Przyspieszenie

Rysunek 6: Analiza wydajności algorytmu w wersji 3. dla dużego rozmiaru kubełka.